

How Inspections Work

This guide explains, in plain language, how an inspection collects information in the BlockPro app and the rules the app applies as the inspector fills it in. Everything in this document describes what the app actually does today, as built.

It sits alongside the [Login Flow](#) and [Data Refresh Flow](#) guides.

The short version

- An inspection is tied to **one asset** (a door, an extinguisher, etc.).
 - The inspector is shown every question for that asset in one scrollable list, grouped visually by chapter.
 - Each question has two rules attached: **what answers are allowed** and **when a photo is required**.
 - The app enforces those rules at submit time — you can't submit until every structured question has an answer and every required photo has been attached.
 - When you tap **Submit**, the photos are uploaded first, then the inspection is sent to the server in one go.
-

What the inspector sees

When an inspection is opened for an asset:

- All of that asset's questions are loaded and shown in a single scrollable list.
- Questions are grouped under **chapter headings**.
- Within each chapter, questions appear in a fixed order set by the server.
- Each question shows: a number, the question text, an optional description, the answer choices, and (when the rules say so) a photo section.
- Any **existing remedials** recorded against that question in earlier inspections are shown in a highlighted box for context. They're read-only — the inspector does not edit them here.

The inspection does **not** page through questions one at a time. The inspector scrolls the full list and answers in any order.

Question types

Every question has an **answer type** that controls which buttons the inspector sees. The app supports four structured answer types:

Answer type	Choices shown
Yes / No	Yes, No
Yes / No / N/A	Yes, No, N/A
Satisfactory / Unsatisfactory	Satisfactory, Unsatisfactory
Satisfactory / Unsatisfactory / N/A	Satisfactory, Unsatisfactory, N/A

Only one answer can be selected per question.

If a question arrives from the server without a recognised answer type, the app falls back to a **free-text input** for that question.

Note: "N/A" is treated as a valid answer, not as "skip". It counts as answered. The app has no skip-this-question feature — questions that might not apply to every asset are handled by giving them an N/A option.

Rule 1 — What answers are allowed

This is the answer type described above. The inspector can only choose from the buttons shown. There is no custom-text override for structured questions; if the question has a defined answer type, one of the provided choices must be picked.

Rule 2 — When a photo is required

Every question that has a photo rule falls into one of two categories:

Photo rule	When a photo is required
Always	A photo must be attached regardless of the answer.
Only when unsatisfactory	A photo must be attached only if the answer is a negative one.

What counts as "negative":

Answer type	Treated as negative
Yes / No (and Yes / No / N/A)	No
Satisfactory / Unsatisfactory (and the N/A variant)	Unsatisfactory

N/A is never treated as negative, so choosing N/A never triggers a photo requirement under the "Only when unsatisfactory" rule.

How the photo section behaves

- If the rule is **Always**, the photo section is shown from the start and requires at least one photo.
 - If the rule is **Only when unsatisfactory**, the photo section only appears once a negative answer is selected.
 - If the inspector changes a negative answer to a non-negative one, **any photos already attached to that question are cleared**. (The app discards them because they are no longer required.)
 - Photos can be taken with the camera or picked from the gallery, and multiple photos can be attached per question.
-

Rule 3 — Order of questions

The order in which questions appear is controlled by the server, not by the app:

- Chapters are shown in the order set by the server.
- Within each chapter, questions are shown in the order set by the server.

The app does not re-order, hide, or filter questions based on asset type, prior answers, or any other local logic.

Rule 4 — What must be answered before submitting

When the inspector taps **Submit Inspection**, the app validates each question in order and blocks submission if any of the following is true:

1. A question with a structured answer type has **no answer selected**.
2. A question where a photo is currently required has **no photos attached**.

If validation fails, the app jumps to the **first** question that failed and shows one of:

- *"Please answer question N"* — an answer is missing.
- *"Photo required for question N"* — the answer is fine but a required photo is missing.

Free-text questions (the fallback type) are treated as optional — they can be left blank.

What the app does not enforce

To be clear about the boundaries of the rules the app applies today:

- **No conditional / skip logic.** One question's answer never hides, shows, or changes another question. The only built-in way to mark a question as not applicable is the N/A option, when the question's answer type includes it.
- **No pass/fail scoring.** The app does not calculate a score or an overall pass/fail for the inspection. Answers are recorded as-is.
- **No automatic remedial creation.** Selecting a negative answer does **not** raise a remedial inside the app. Existing remedials from past inspections are shown as read-only context only; any new remedial flow, if one exists, happens on the server after submission.
- **No draft / resume.** The inspection is only held in the app's memory until Submit is tapped. Closing the screen, closing the app, or a crash will lose unsubmitted answers.

These aren't oversights in this document — they describe the behaviour that currently exists in the code.

What happens when Submit is tapped

The submission is a two-step process:

Step 1 — Upload photos

For every photo attached to every answer, the app:

1. Reads the photo file and encodes it.
2. Sends it to the `app_upload-image` API, together with the asset's id and the filename.
3. Collects the `image_id` the server returns.

Photos are uploaded one at a time. **If a single photo upload fails, the app logs the failure and carries on** — the rest of the submission still goes through, but that photo won't be attached to the inspection.

Step 2 — Submit the inspection

Once photos have been attempted, the app sends the inspection to the `app_completed-inspection` API with:

- The **asset id**.
- A list of **answers** — each one containing the question's **text** and the **answer text** the inspector selected.
- The **photo ids** collected from Step 1 (if there were any).

Note that the submission sends the **question text**, not a question id. The server matches answers back to questions by the wording the inspector was shown.

Result

- On success, the app shows "*Inspection submitted successfully*" and returns the inspector to the asset list.
- On failure, the error is shown in the submit bar and the inspector stays on the form. They can correct the issue and retry.

What gets remembered, and what doesn't

Thing	Held where	Survives app restart?
The checklist (chapters + questions) for the asset	Device database	Yes
Existing remedials shown for context	Device database	Yes
The inspector's in-progress answers	In memory only	No
Photos the inspector has attached but not yet submitted	In memory only	No
Submitted inspection	Server (and re-synced down as a completed inspection)	Yes

For more on how the checklist itself is kept up to date — and the interaction between refresh and in-progress inspections — see the [Data Refresh Flow](#) guide.

Quick reference — the rules the app enforces

Rule	Effect
Answer type on a question	Controls which answer buttons appear (Yes/No, Yes/No/N/A, Satisfactory/Unsatisfactory, Satisfactory/Unsatisfactory/N/A).
Photo rule = Always	A photo must be attached before submission.
Photo rule = Only when unsatisfactory	A photo must be attached only if a negative answer (No / Unsatisfactory) is selected. N/A does not trigger this.
Changing away from a negative answer	Any photos attached to that question are cleared.
Question order	Server-defined, per chapter. The app does not reorder.
Submit validation	Every structured question must have an answer. Every currently-required photo must be attached. Otherwise the submit is blocked and the app jumps to the first offending question.
Photo upload failure	Logged and skipped. The rest of the inspection is still submitted.

APIs used during an inspection

API	Purpose
-----	---------

app_upload-image	Uploads a single photo tied to the asset being inspected. Returns an <code>image_id</code> that the app attaches to the submission. Called once per photo.
app_completed-inspection	Submits the completed inspection — the asset id, the list of answers (question text + answer text), and any photo ids produced by the uploads.